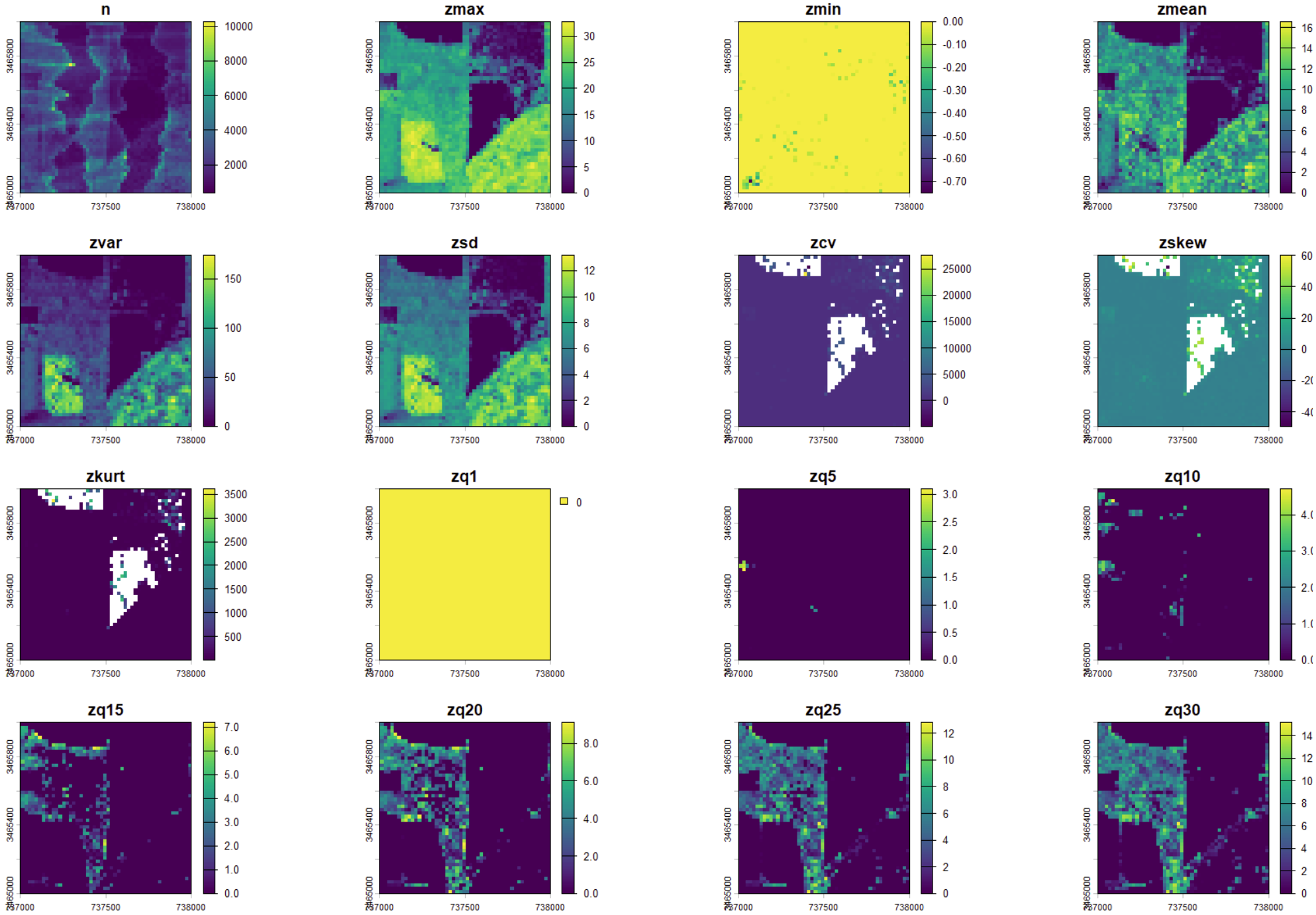
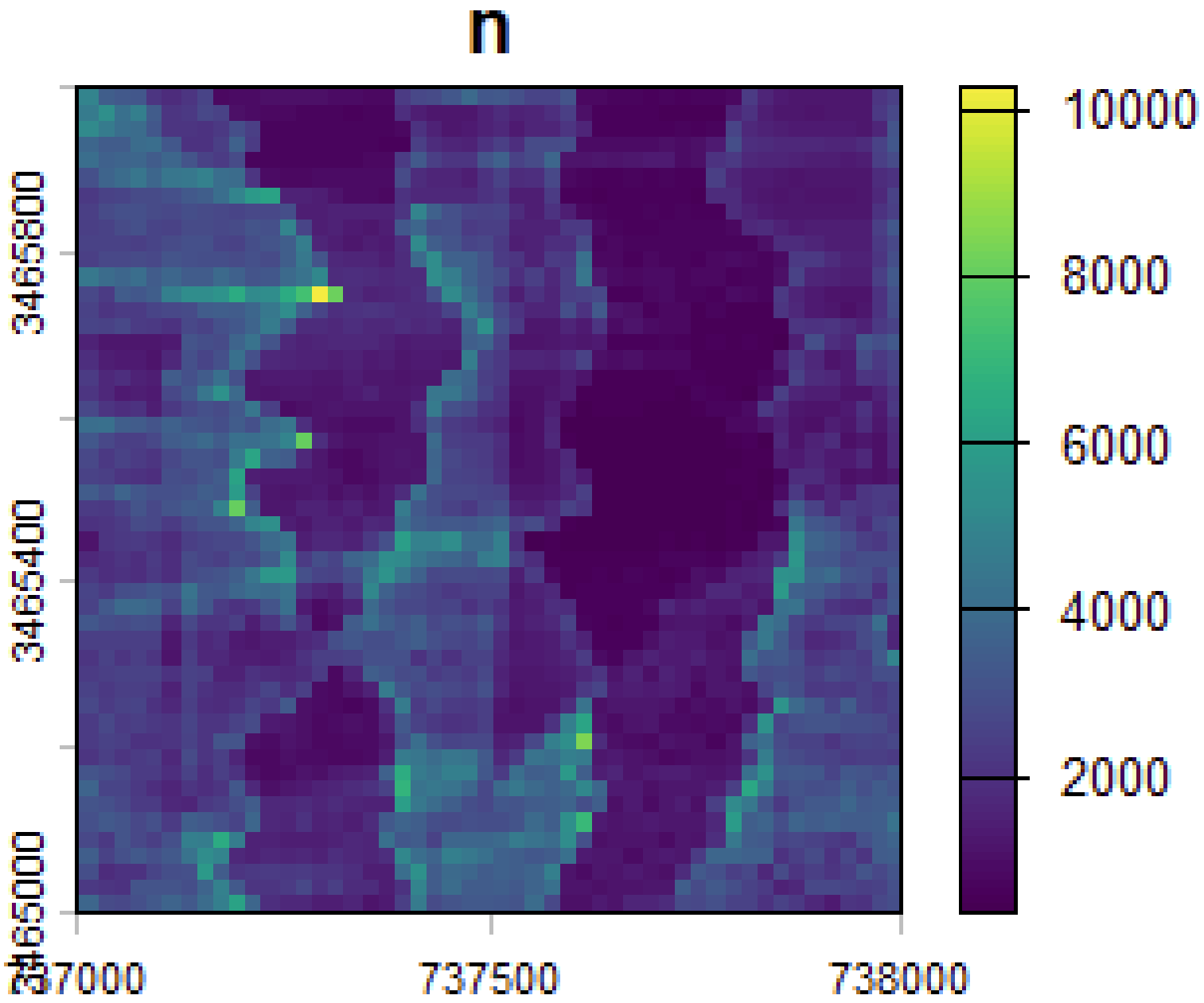
Reduce density but preserve structural information

Master_metrics_function.R

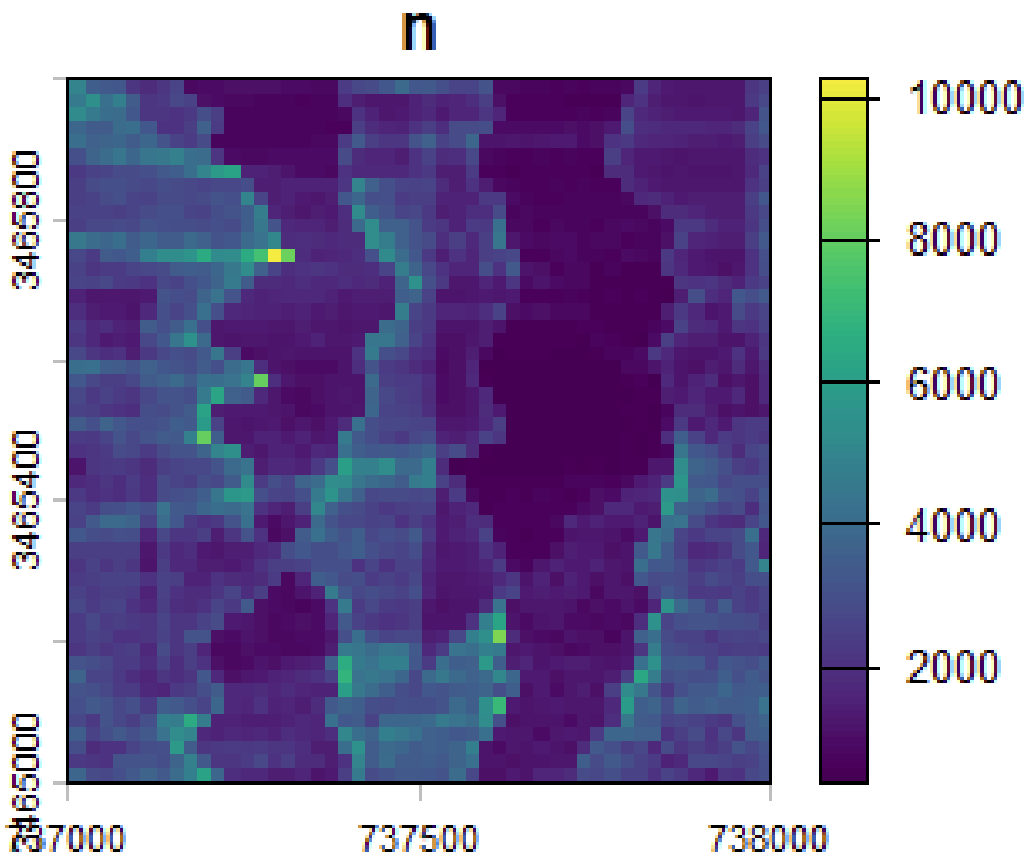
Source on Save

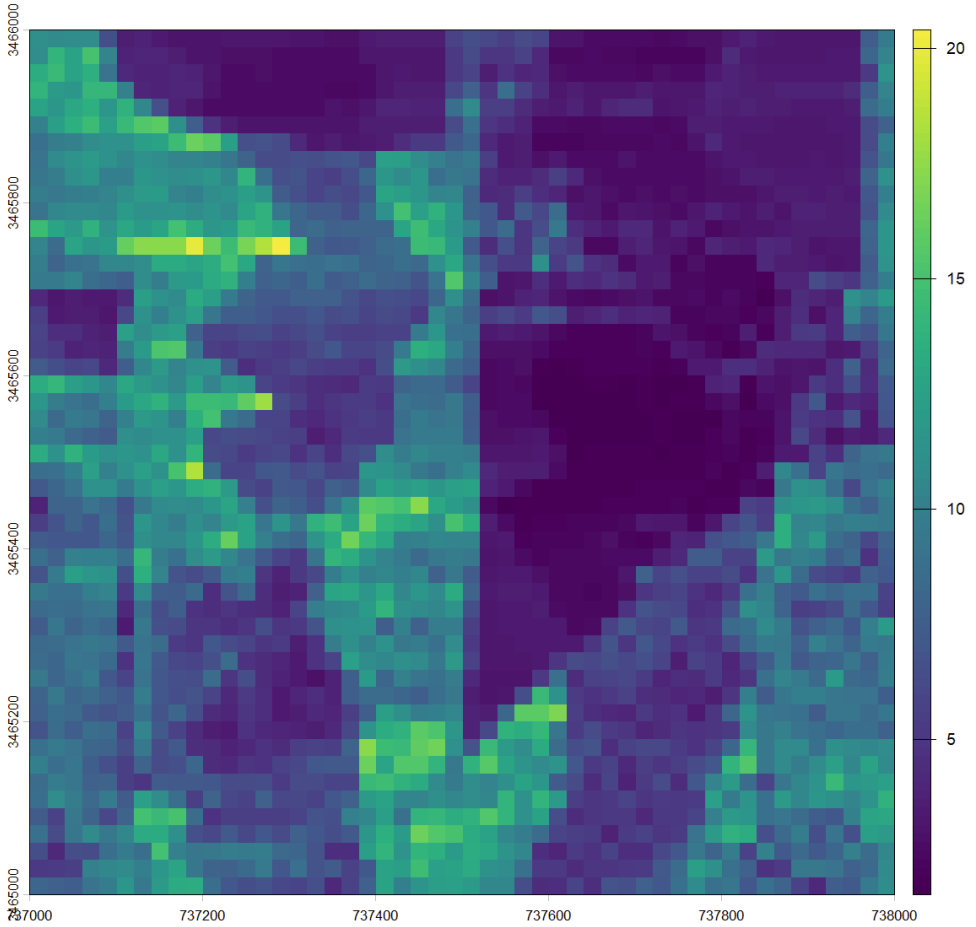
```
1  library(lidR)
2  library(terra)
3  library(lidRmetrics)
4  library(Lmoments)
5  install.packages("geometry")
6
7  source("Master_metrics_function.R")
8
9  las <- readLAS("NEON_lidar_tile.laz", filter = "-keep_random_fraction 0.5")
10
11 las <- readLAS("NEON_lidar_tile.laz", filter = "-thin_with_voxel 0.25")
12
13 table(las$Classification)
14 # Appears to be some buildings labeled 6 -> LASBUILDING
15 las <- filter_poi(las, !Classification %in% c(LASNOISE,LASBUILDING))
16 las <- normalize_height(las, algorithm = tin())
17
18 metric_stack <- pixel_metrics(las, ~master_metrics(X, Y, Z, Intensity, ReturnNumber, NumberOfRetur
19                                res = 20)
20 names(metric_stack)
21 #108 different layers
22 plot(metric_stack)
```


n
zmax
zmin
zmean
zvar
zsd
zcv
zskew
zkurt
zq1
zq5
zq10
zq15
zq20
zq25
zq30
3465800
3465400
3465000
737000
737500
738000

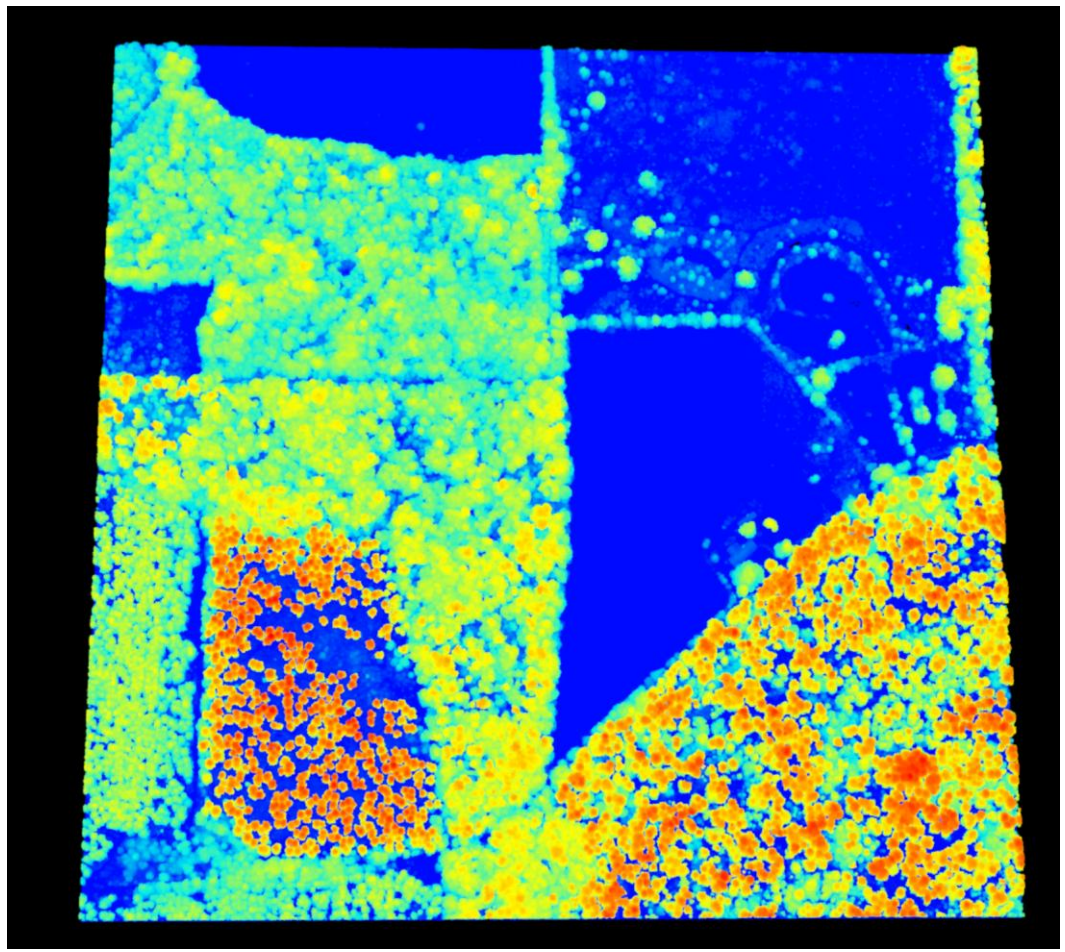

n
10000
8000
6000
4000
2000
3465800
3465400
3465000
737000
737500
738000

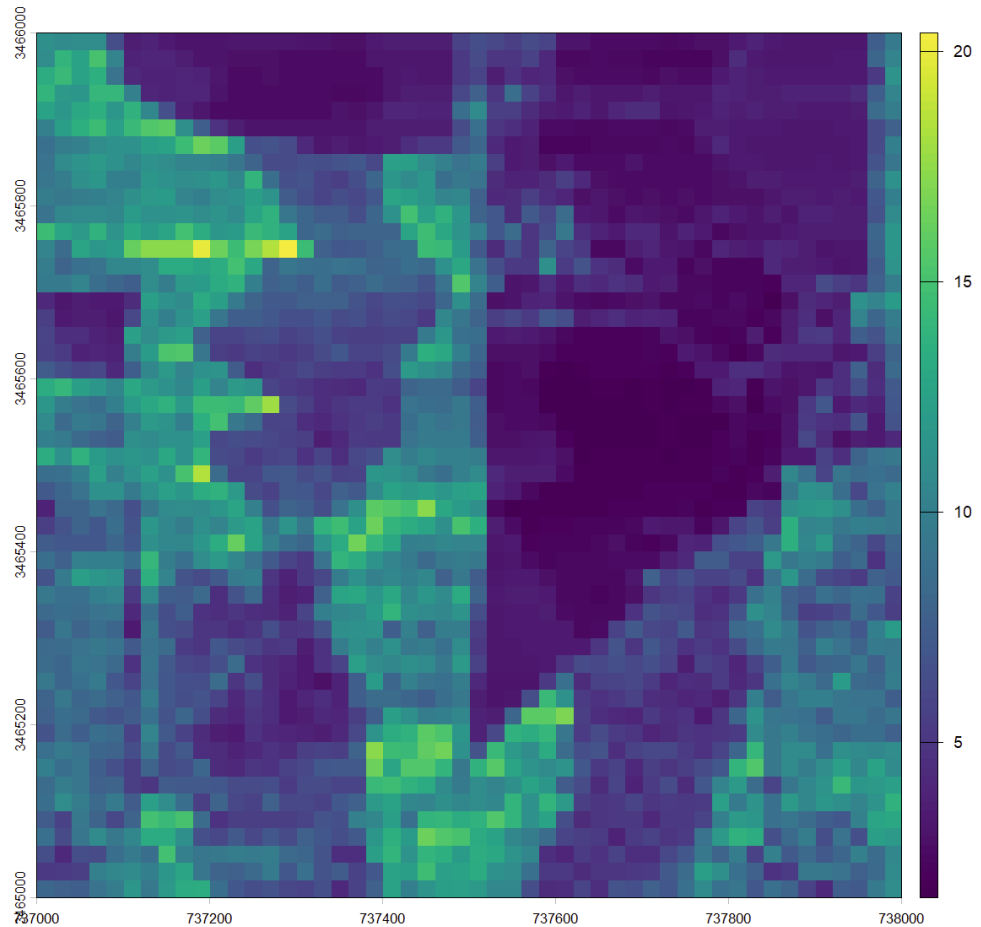
```r
#las <- readLAS("NEON_lidar_tile.laz", filter = "-keep_random_fraction 0.5")
las <- readLAS("NEON_lidar_tile.laz", filter = "-thin_with_voxel 0.5")

x = pixel_metrics(las, ~length(Z)/(20*20), res=20)
plot(x)
```

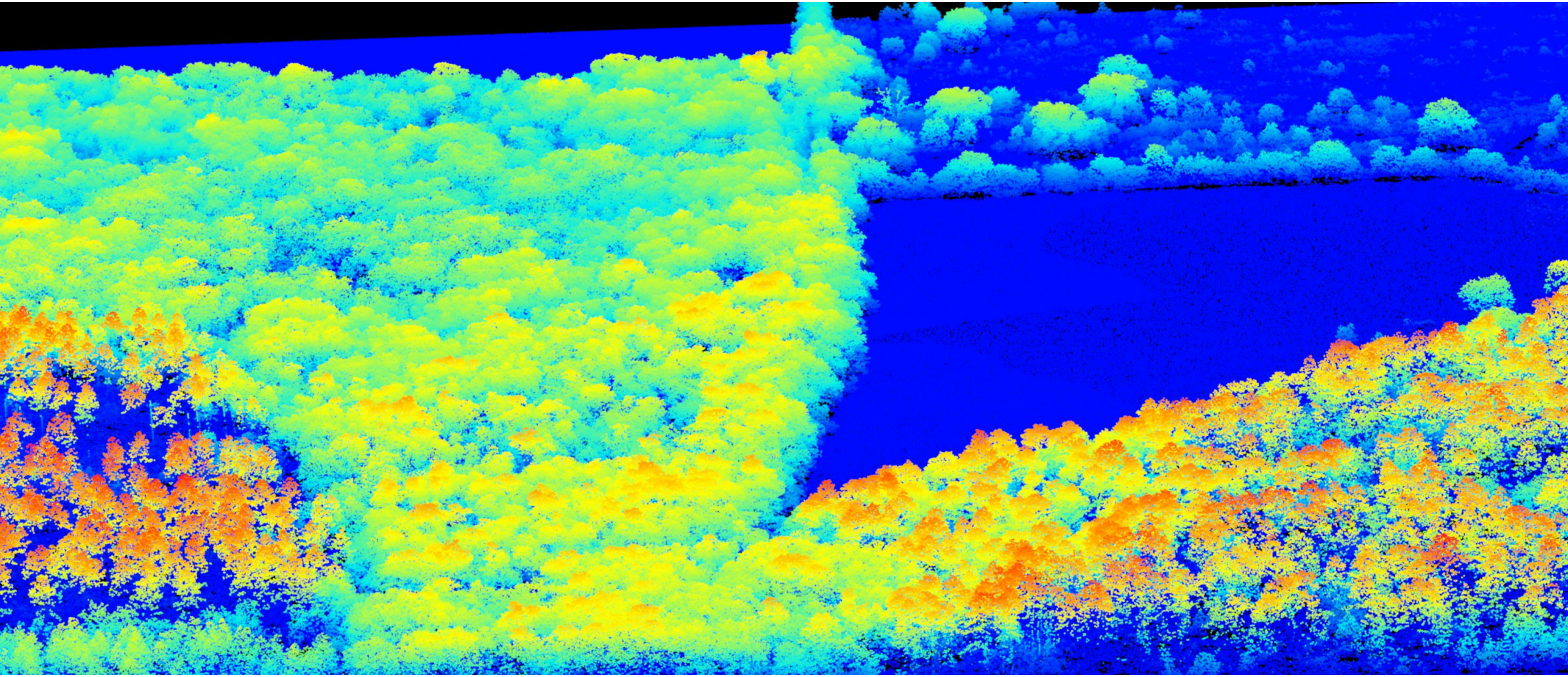


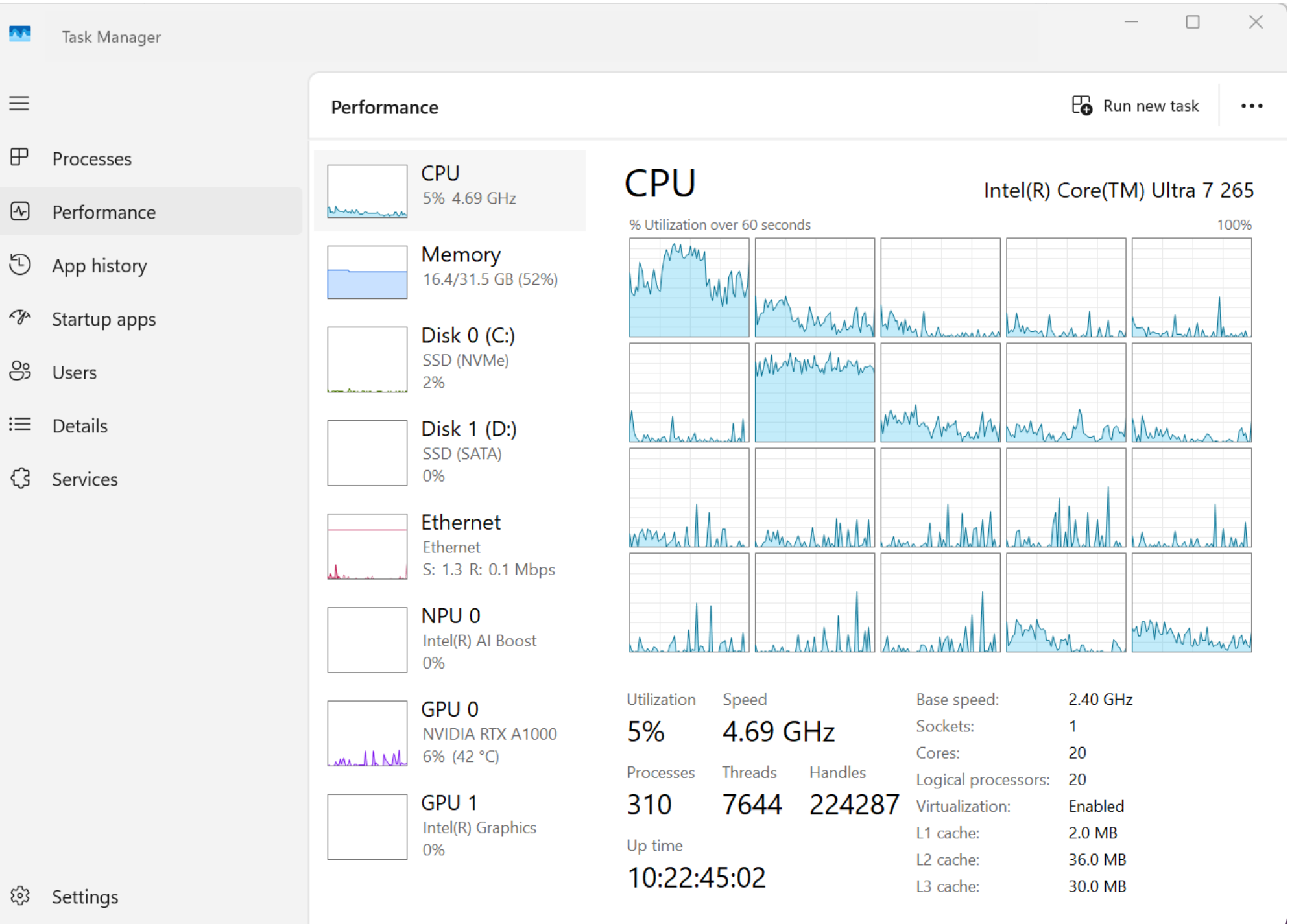


```
8
9  #las <- readLAS("NEON_lidar_tile.laz", filter = "-keep_random_fraction 0.5")
10 las <- readLAS("NEON_lidar_tile.laz", filter = "-thin_with_voxel 0.5")
11
12 x = pixel_metrics(las, ~length(Z)/(20*20), res=20)
13 plot(x)
```









Run singly and check memory

```r
library(future)

source("Master_metrics_function.R")

# --- parallel backend ---
plan(multisession, workers = 3)

# --- catalog setup ---
ctg <- readLAScatalog('I:/neon/2019/lidar/ClassifiedPointCloud/')
fns <- ctg@data$filename[1:5]
ctg <- readLAScatalog(fns)
plot(ctg)

opt_chunk_alignment(ctg) <- c(0, 0)
opt_filter(ctg)          <- '-thin_with_voxel 0.5'
opt_chunk_buffer(ctg)    <- 20

plot(ctg, chunk_pattern = TRUE)

myfun <- function(las, res) {
  las          <- filter_poi(las, !Classification %in% c(LASNOISE, LASBUILDING))
  las          <- normalize_height(las, algorithm = tin())
  metric_stack <- pixel_metrics(
    las,
    ~master_metrics(X, Y, Z, Intensity, ReturnNumber, NumberOfReturns),
    res = res
  )
  return(metric_stack)
}

output <- lidR::catalog_map(ctg, myfun)
```

Overwrite names for memory mgmt

```
24  myfun <- function(las, res) {
25    las          <- filter_poi(las, !Classification %in% c(LASNO
26    las          <- normalize_height(las, algorithm = tin())
27    metric_stack <- pixel_metrics(
```

plot 8

VARIABLES

R 4.5.1 | Filter

VALUES

chunk	<S4 class 'LAScluster' [package "lidR"] with 16 slots>	LAScluster
ctg	<S4 class 'LAScatalog' [package "lidR"] with 6 slots>	LAScatalog
ctg_full	<S4 class 'LAScatalog' [package "lidR"] with 6 slots>	LAScatalog
fns	"I:\neon\2019\lidar\ClassifiedPointCloud\NEON_D03_JERC_DP1_731000_3459000_classified_point_clou...	str [5]
las	<S4 class 'LAS' [package "lidR"] with 4 slots>	LAS
output	<S4 class 'SpatRaster' [package "terra"] with 1 slot>	SpatRaster

FUNCTIONS

master_metrics	function (x, y, z, i, ReturnNumber, NumberOfReturns, zmin = NA, threshold = c(2, 5), dz = 1, interval_cou...	
myfun	function (las, res)	

Task Manager

Performance

Run new task

Processes
Performance
App history
Startup apps
Users
Details
Services
Settings

CPU
20% 4.69 GHz

Memory
24.7/31.5 GB (78%)

Disk 0 (C:)
SSD (NVMe)
5%

Disk 1 (D:)
SSD (SATA)
0%

Ethernet
Ethernet
S: 216 R: 152 Kbps

NPU 0
Intel(R) AI Boost
0%

GPU 0
NVIDIA RTX A1000
3% (39 °C)

GPU 1
Intel(R) Graphics
0%

CPU

Intel(R) Core(TM) Ultra 7 265

% Utilization over 60 seconds

100%

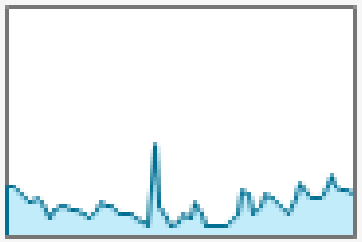
Utilization	Speed
20%	4.69 GHz

Processes	Threads	Handles
327	7800	227183

Up time
10:23:11:27

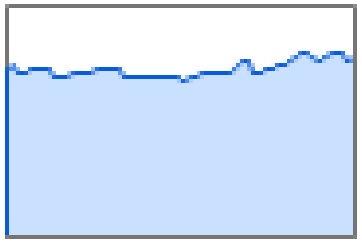
Base speed:	2.40 GHz
Sockets:	1
Cores:	20
Logical processors:	20
Virtualization:	Enabled
L1 cache:	2.0 MB
L2 cache:	36.0 MB
L3 cache:	30.0 MB

Performance



CPU

20% 4.69 GHz



Memory

24.7/31.5 GB (78%)

Disk 0 (C:)

SSD (NVMe)

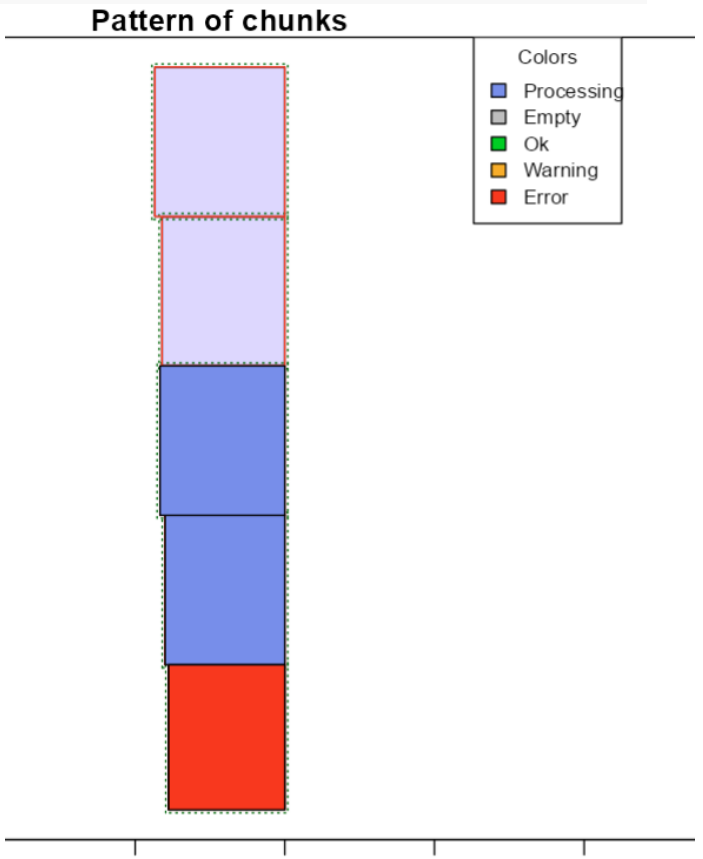
Error:

! scan() expected 'a real', got 'IllegalArgumentException:'

Hide Traceback ✧Fix ✧Explain

```
1. └─lidR::catalog_map(ctg, myfun, res = 20) at Github/codeblitz4/Master_metrics_testing_catalog.R:35:1
2.   └─lidR::catalog_apply(ctg, FUN, ..., .options = .options)
3.     └─lidR::engine_apply(...)
```

>

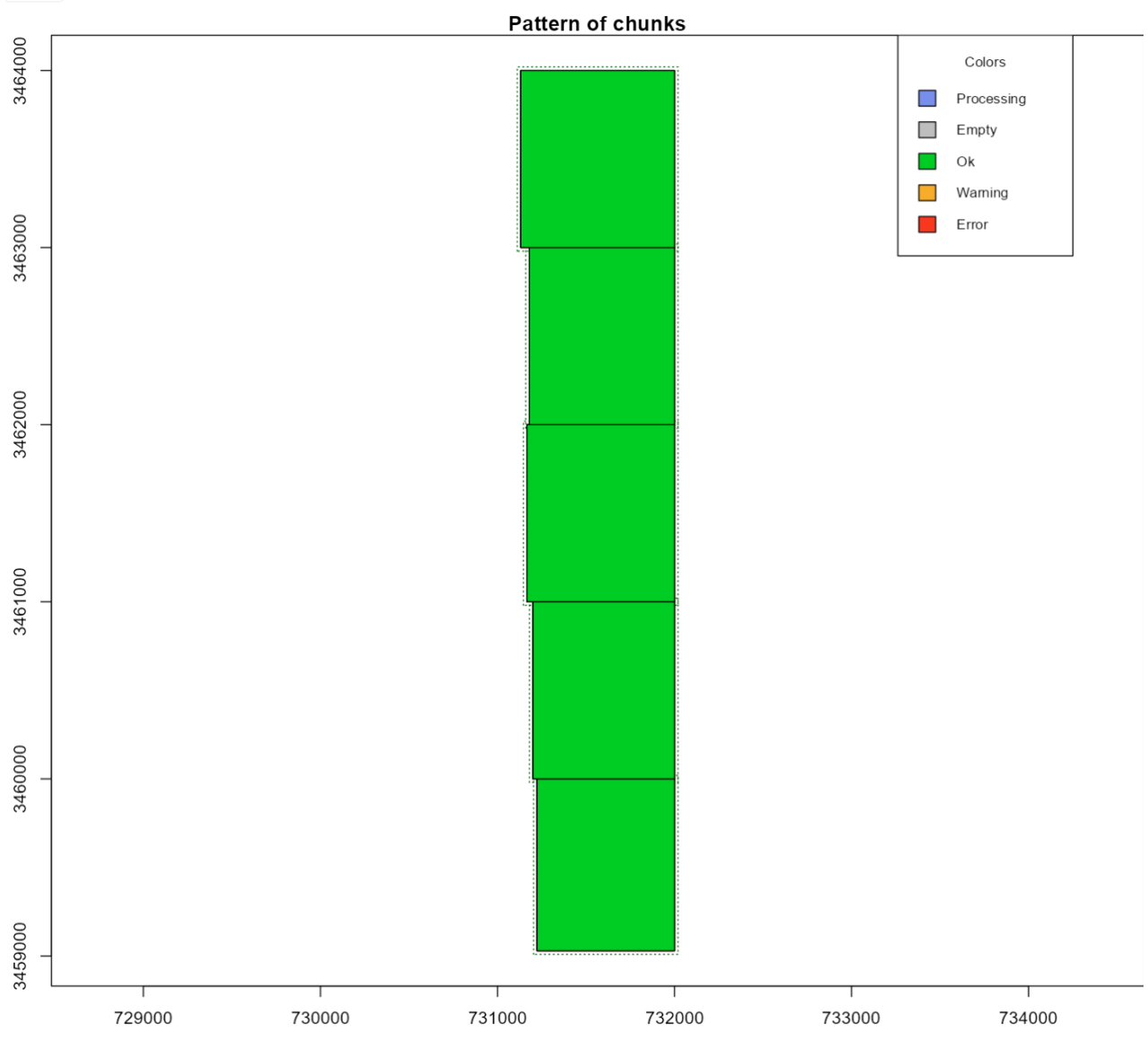


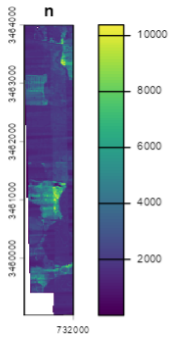
```
36
37  # HOW TO TROUBLESHOOT AND RUN A SIGNLE CHUNK.
38  chunks = engine_chunks(ctg)
39  chunks[1]
40  chunk = chunks[[1]]
41  las = readLAS(chunk)
42  output = myfun(las, res=20)
43  plot(output)
44
```

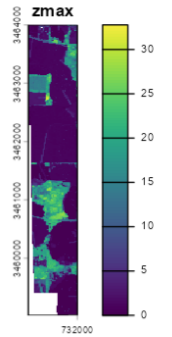
Master_metrics_function.R > master_metrics

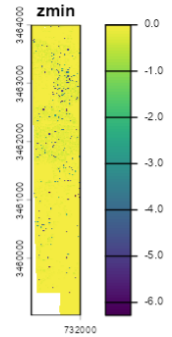
```
90  master_metrics <- function(x, y, z, i, ReturnNumber, NumberOfReturns,
117                            vox_size = vox_size, zmin = zmin)
118
119   m_kde    <- lidRmetrics::metrics_kde(z = z, zmin = zmin)
120
121   m_HOME   <- lidRmetrics::metrics_HOME(z = z, i = i, zmin = zmin)
122
123   mt       <- lidRmetrics::metrics_texture(x = x, y = y, z = z,
124                                            pixel_size = pixel_size,
125                                            zmin = zmin,
126                                            chm_algorithm = chm_algorithm)
127
128   m <- c(m_set1, m_echo, m_echo2, m_rumple, m_vox, m_kde, m_HOME, mt)
129   return(m)
130 }
```

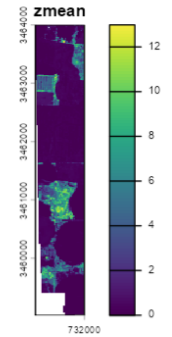
```
90  master_metrics <- function(x, y, z, i, ReturnNumber, NumberOfReturns,
117                            vox_size = vox_size, zmin = zmin)
118
119   m_kde    <- lidRmetrics::metrics_kde(z = z, zmin = zmin)
120
121   m_HOME   <- lidRmetrics::metrics_HOME(z = z, i = i, zmin = zmin)
122
123+  texture_na <- list(glcm_contrast = NA, glcm_dissimilarity = NA,
124+                     glcm_homogeneity = NA, glcm_ASM = NA,
125+                     glcm_entropy = NA, glcm_mean = NA,
126+                     glcm_variance = NA, glcm_correlation = NA)
127+
128+  n_cells <- nrow(unique(data.frame(floor(x / pixel_size), floor(y / pixel_size))))
129+
130+  mt <- if (n_cells >= 3) {
131+    tryCatch(
132+      lidRmetrics::metrics_texture(x = x, y = y, z = z, pixel_size = pixel_size,
133+                                   zmin = zmin, chm_algorithm = chm_algorithm),
134+      error = function(e) texture_na
135+    )
136+  } else {
137+    texture_na
138+  }
139
140   m <- c(m_set1, m_echo, m_echo2, m_rumple, m_vox, m_kde, m_HOME, mt)
141   return(m)
142 }
```

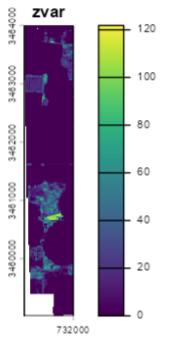

Pattern of chunks
Colors
Processing
Empty
Ok
Warning
Error
3464000
3463000
3462000
3461000
3460000
3459000
729000
730000
731000
732000
733000
734000

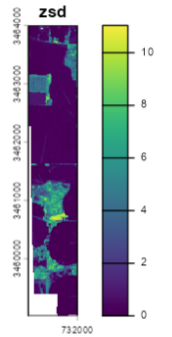

n
3464000
3463000
3462000
3461000
3460000
732000
10000
8000
6000
4000
2000

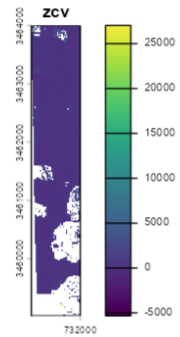

zmax
3464000
3463000
3462000
3461000
3460000
732000
30
25
20
15
10
5
0

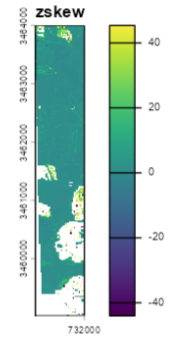

zmin
3464000
3463000
3462000
3461000
3460000
732000
0.0
-1.0
-2.0
-3.0
-4.0
-5.0
-6.0

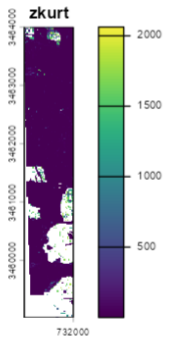

zmean
3464000
3463000
3462000
3461000
3460000
732000
12
10
8
6
4
2
0

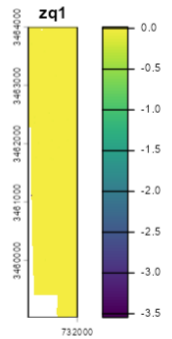

zvar
3464000
3463000
3462000
3461000
3460000
732000
120
100
80
60
40
20
0

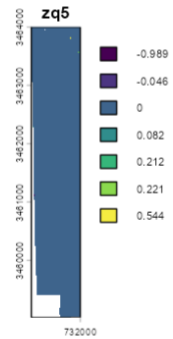

zsd
3464000
3463000
3462000
3461000
3460000
732000
10
8
6
4
2
0

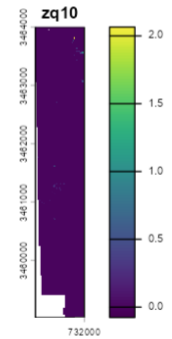

zcv
3464000
3463000
3462000
3461000
3460000
732000
25000
20000
15000
10000
5000
0
-5000

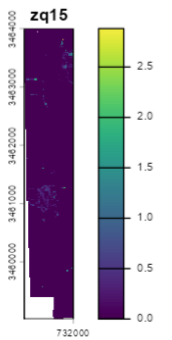

zskew
3464000
3463000
3462000
3461000
3460000
732000
40
20
0
-20
-40

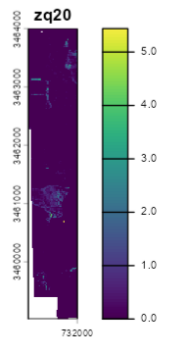

zkurt
3464000
3463000
3462000
3461000
3460000
732000
2000
1500
1000
500

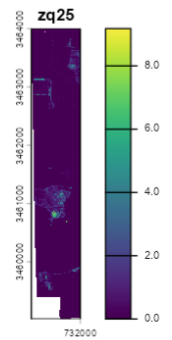

zq1
3464000
3463000
3462000
3461000
3460000
732000
0.0
-0.5
-1.0
-1.5
-2.0
-2.5
-3.0
-3.5

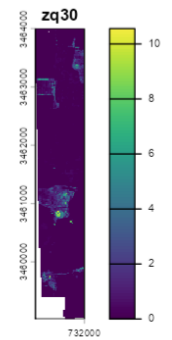

zq5
3464000
3463000
3462000
3461000
3460000
732000
-0.989
-0.046
0
0.082
0.212
0.221
0.544


zq10
3464000
3463000
3462000
3461000
3460000
732000
2.0
1.5
1.0
0.5
0.0


zq15
3464000
3463000
3462000
3461000
3460000
732000
2.5
2.0
1.5
1.0
0.5
0.0

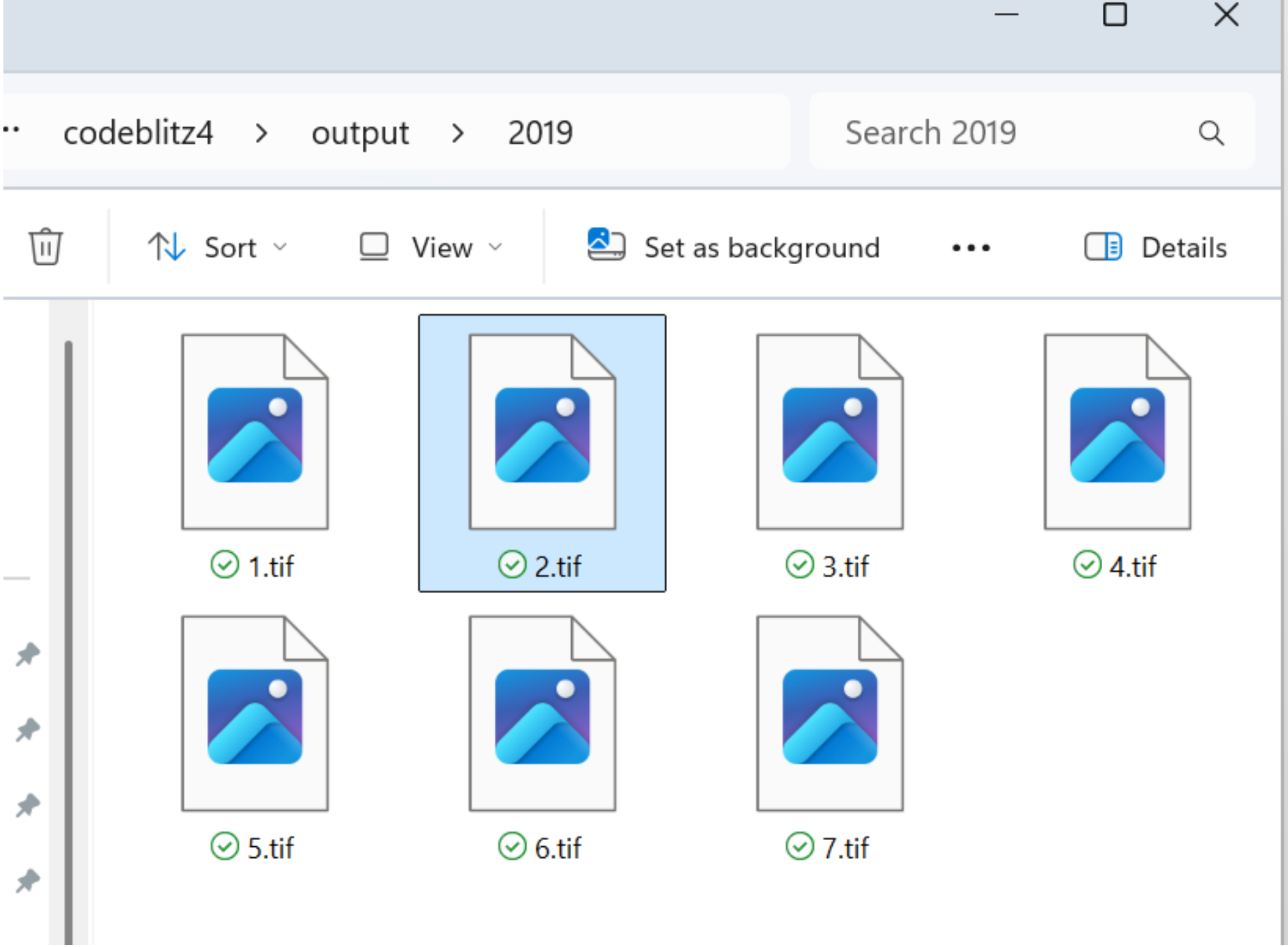

zq20
3464000
3463000
3462000
3461000
3460000
732000
5.0
4.0
3.0
2.0
1.0
0.0


zq25
3464000
3463000
3462000
3461000
3460000
732000
8.0
6.0
4.0
2.0
0.0


zq30
3464000
3463000
3462000
3461000
3460000
732000
10
8
6
4
2
0

```r
# ============================================================================
# FULL CATALOG RUN — processes all tiles, saves each to disk, mosaics at end.
# catalog_apply writes one raster per tile via opt_output_files; terra::mosaic
# merges them into a single wall-to-wall stack.
# ============================================================================

ctg_full <- readLAScatalog('I:/neon/2019/lidar/ClassifiedPointCloud/')
opt_chunk_alignment(ctg_full) <- c(0, 0)
opt_filter(ctg_full)          <- '-thin_with_voxel 0.5'
opt_chunk_buffer(ctg_full)    <- 20
opt_output_files(ctg_full)    <- 'output/2019/{ID}'

plan(multisession, workers = 3)
catalog_map(ctg_full, myfun, res = 20)
#not saving output as anything bc it will output to folder

# mosaic all saved tiles into one raster
tile_files  <- list.files('output/2019', pattern = '\\.tif$', full.names = TRUE)
tile_rasts  <- lapply(tile_files, rast)
mosaic_full <- do.call(mosaic, c(tile_rasts, list(fun = 'mean')))
writeRaster(mosaic_full, 'output/2019_metrics.tif')
```

Save outputs as you go



Save outputs as you go